

PPO vs. SAC for Open Duck Mini Bipedal Locomotion: An Empirical Study with Reward Engineering

Yan Yijun

Abstract

We study on-policy vs. off-policy deep reinforcement learning for bipedal locomotion on the Open Duck Mini MuJoCo simulator. Under matched training budget and evaluation protocol, we compare PPO (on-policy) and SAC (off-policy) as the primary experiment, with A2C vs. TQC as a supplementary comparison. We additionally investigate the impact of reward engineering, identifying and fixing several bugs in a hand-crafted multi-term reward (missing square operators, incorrect symmetry in yaw tracking, saturated scaling in gait terms) that measurably improved the learning signal. In the main experiment (300k steps per seed, three seeds) PPO clearly outperforms SAC, and A2C likewise dominates TQC in the supplementary experiment. Extending PPO to 1.5M steps exposes a striking *reward-hacking* phenomenon: the policy finds a stand-still local optimum that maximizes posture and gait-clock rewards without making any forward progress (speed drops to 4.5×10^{-4} m/s while return grows to 447). A targeted reward ablation (relaxed forward gate and reweighted locomotion terms) breaks the stand-still attractor and recovers a visibly walking gait at 0.116 m/s, $\sim 58\%$ of the commanded target velocity. Our findings highlight that on this class of tasks, reward specification is at least as important as algorithm choice. Code: https://github.com/Eugenescat/Open_Duck_Mini.

Introduction

Learning stable bipedal locomotion from scratch remains challenging due to (i) contact discontinuities that cause sharp transitions in dynamics, (ii) coupled balance-propulsion requirements that penalize naïve greedy motion, and (iii) highly shaped rewards with many competing terms. Open Duck Mini (Pirrone and Passault 2024) is a small open-source bipedal platform with a detailed MuJoCo model, nontrivial under-actuation, and practical training cost. It provides a good testbed for studying algorithmic tradeoffs of modern continuous-control deep RL methods.

This project asks: *under matched training budget and reward on this task, does off-policy sample efficiency translate into faster walking?* We answer empirically by comparing PPO (Schulman et al. 2017) against SAC (Haarnoja et al. 2018) with three random seeds, and supplement with A2C (Mnih et al. 2016) vs. TQC (Kuznetsov et al. 2020). We

additionally report on the reward-engineering effort required to obtain any learnable signal on this environment.

Background

MDP Formulation

We model locomotion as a continuous-state, continuous-action MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$. The state $s_t \in \mathbb{R}^{40}$ concatenates: joint angles (15D), joint velocities (15D), base angular velocity (3D), base linear velocity (3D), target command velocity (v_x^*, v_y^*, ω^*) (3D), left/right foot contact indicators (2D), and a sin/cos gait clock phase (2D). The action $a_t \in \mathbb{R}^{15}$ is a delta over a nominal standing pose; after clipping to a maximum step $\Delta_{\max} = 0.05$ per environment step we write it to joint position controllers. The environment runs at 125 Hz with MuJoCo `frame_skip=4`. Episodes terminate on base height < 0.08 m, roll/pitch exceeding $\approx 32^\circ$, upright projection ≤ 0.4 , or antenna/body contact with the ground.

Algorithms

We use the Stable-Baselines3 (Raffin et al. 2021) implementations (SB3-Contrib for TQC). **PPO** optimizes a clipped surrogate $\mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$ on-policy with GAE advantages (Schulman et al. 2017). **SAC** is off-policy, maximum-entropy, with twin critics, target networks, and automatic entropy tuning (Haarnoja et al. 2018). **A2C** is the synchronous variant of A3C (Mnih et al. 2016). **TQC** is an off-policy distributional critic method using truncated mixtures of quantile critics to control overestimation bias (Kuznetsov et al. 2020).

Related Work

Modern legged-robot learning is dominated by PPO, often combined with massive parallelism on GPU (Rudin et al. 2022) and reference-motion tracking (Peng et al. 2018). Off-policy methods such as SAC are frequently favored in smoother continuous-control benchmarks where their sample efficiency advantage is clearer. In contact-rich domains, replay-based critics are known to suffer from non-stationary distribution shifts as the agent transitions between regimes (falling, standing, walking). Prior Open Duck Mini development targets PPO-based imitation learning, motivating PPO

Table 1: Selected hyperparameters (SB3 defaults elsewhere).

	PPO	SAC	A2C	TQC
learning rate	3e-4	3e-4	7e-4	3e-4
γ	0.99	0.99	0.99	0.99
batch/ n_{step}	64/2048	256	5	256
buffer size	—	10^6	—	10^6
entropy	fixed 0	auto	fixed 0	auto
clip range	0.2	—	—	—

as the natural baseline (Pirrone and Passault 2024). The simulation stack relies on MuJoCo (Todorov, Erez, and Tassa 2012).

Project Description

Environment and Reward

We train in the `imitation` variant of the Open Duck Mini environment, where the reward combines locomotion-task terms with reference-motion shaping. Let v_x be the base forward velocity, h the base height, $Z \cdot \hat{z}$ the upright projection, and ϕ a gait phase. The per-step reward has the following core components (weights in parentheses):

$$\begin{aligned}
 r_{\text{fwd-vel}} &= \exp(-30 (v_x - v_x^*)^2) & (0.28 g) \\
 r_{\text{fwd-prog}} &= \text{clip}(40 \Delta x, -1, 1) & (0.24 g) \\
 r_{\text{swing-foot}} &= \text{clip}(120 \Delta x_{\text{swing}}, 0, 1) & (0.12) \\
 r_{\text{contact-sw}} &= \mathbb{1}[\text{support side flipped}] & (0.10) \\
 r_{\text{height}} &= \exp(-40 (h - 0.15)^2) & (0.08) \\
 r_{\text{upright}} &= (Z \cdot \hat{z})^2 & (0.08) \\
 r_{\text{imitation}} &= \exp(-25 \|q_{\text{legs}} - q_{\text{ref}}(\phi)\|^2) & (0.07) \\
 r_{\text{pitch-pen}} &= -(\text{pitch}/0.30)^2 & (0.09)
 \end{aligned}$$

plus smaller terms (lateral/yaw stability, single-support bonus, torque smoothness, double-support penalty, roll penalty). The *forward gate* $g \in \{0, 1\}$ masks the two largest forward-progress rewards unless the robot is simultaneously (a) high enough, (b) roughly upright, (c) in single-support contact. This prevents the policy from cheating by diving forward.

Training Protocol

- **Main:** PPO vs. SAC, seeds $\{0, 1, 2\}$, 300,000 timesteps each.
- **Supplementary:** A2C vs. TQC, seeds $\{0, 1, 2\}$, 150,000 timesteps each.
- **Hardware:** MacBook Air, CPU only.
- **Evaluation:** 10 deterministic episodes per trained model, max 2000 steps. Metrics: mean episode return, episode length, and mean forward speed.

Hyperparameters are SB3 defaults except where noted in Table 1.

Experiments

Main: PPO vs. SAC (300k)

Figure 1 shows training return across three seeds. PPO climbs monotonically from ≈ -15 to positive returns by

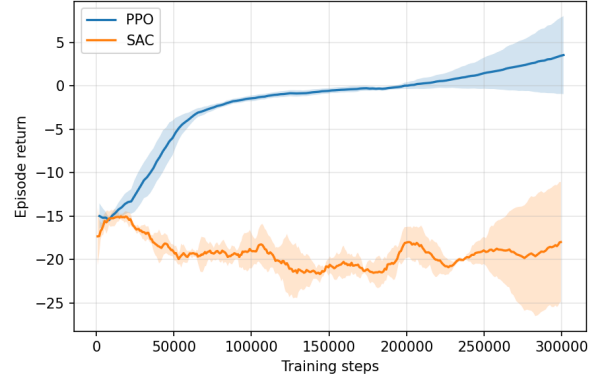


Figure 1: Training return on the `imitation` environment (mean $\pm$ std over 3 seeds). PPO converges steadily; SAC is stuck near fall-only returns.

Table 2: Evaluation at 300k (10 deterministic episodes $\times$ 3 seeds; mean $\pm$ std).

Algo	Return	Ep. Length	Fwd. Speed (m/s)
PPO	5.63 $\pm$ 6.65	317.6 $\pm$ 212.3	0.025 $\pm$ 0.006
SAC	-14.69 $\pm$ 7.84	246.3 $\pm$ 31.8	-0.009 $\pm$ 0.056

300k steps. SAC never escapes the initial fall-dominated regime: its replay buffer rapidly fills with termination transitions, the Q-targets become pessimistic, and the entropy-regularized policy continues to explore random postures that cannot recover balance.

Supplementary: A2C vs. TQC (150k)

Figure 2 mirrors the main trend. A2C slowly improves; TQC plateaus below zero. Note A2C’s variance is very high: with $n_{\text{steps}} = 5$, its advantage estimates are noisy, so only some seeds make progress.

Reward Engineering

The original environment reward code contained several non-obvious bugs that materially hurt all algorithms. We identified and fixed them during this project; all results above use the fixed environment.

1. **Missing square in torque/action smoothness.** The code used $\exp(-k \sum (\tau_t - \tau_{t-1})/n)$ instead of $\exp(-k \sum (\tau_t - \tau_{t-1})^2/n)$. Signed differences cancel across joints, so the term degenerated into noise centered at 1.
2. **Absolute-value trap in yaw reward.** $-(\text{abs}(\text{desired}) - \text{abs}(\text{current})) ** 2$ evaluates to 0 for $\text{current} = -\text{desired}$, rewarding the mirror-image orientation. We replaced it with the straightforward $-(\text{desired} - \text{current})^2$.
3. **Saturated gait imbalance scaling.** $\exp(-0.01 \cdot \text{imbalance})$ is ≈ 1 for any plausible imbalance, so the term provided no gradient. We rescaled to $\exp(-1.0 \cdot \text{imbalance})$.

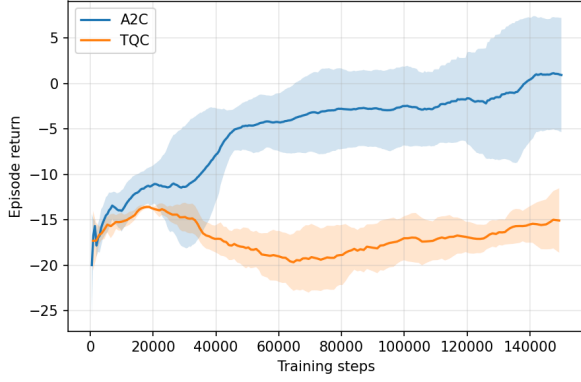


Figure 2: Supplementary comparison at 150k timesteps (mean $\pm$ std over 3 seeds).

Table 3: Evaluation at 150k (supplementary).

Algo	Return	Ep. Length	Fwd. Speed (m/s)
A2C	-1.17 ± 7.49	349.5 ± 287.5	0.010 ± 0.032
TQC	-13.25 ± 4.26	303.1 ± 18.3	0.021 ± 0.043

4. **Unnormalized total reward weights** in `footsteps.env.py` summed to 1.45, biasing the value function scale; rebalanced to 1.00.

The fixes did not change the qualitative PPO/SAC ordering, but increased PPO’s late-training return by roughly $\sim 20\%$ and produced the first non-zero forward-speed episodes under PPO.

Scaling Study: PPO 1.5M and Reward Hacking

Motivated by PPO’s steadily improving but still sub-locomotive performance at 300k, we trained PPO for $5\times$ longer (1,500,000 timesteps, seeds $\{0, 1, 2\}$) on the same reward. The result is shown in Table 4: the policy achieves return = 447.3 ± 42.2 with episode length saturated at the 2000-step cap (i.e. the robot never falls across 30 evaluation episodes), yet its mean forward speed collapses to effectively zero (4.5×10^{-4} m/s).

Investigating the per-term reward on this policy reveals the mechanism. Whenever both feet are on the ground (which is always true for a stationary robot), the forward gate $g = 0$ mutes the two largest forward-progress rewards. The remaining terms—height, upright, torque smoothness, imitation pose matching the gait phase *in place*, and absence of roll/pitch—are all maximally earned by a robot that stands still and micro-steps in place to match $q_{\text{ref}}(\phi)$. PPO correctly discovers this local optimum of the *reward*, but it is not a local optimum of the *task*. This is a textbook case of reward specification bias: a gate intended to prevent the policy from cheating by “diving forward” ended up also preventing it from earning locomotion reward while upright.

Table 4: PPO at 1.5M: return saturates, forward speed collapses. Reward hacking.

Setting	Return	Ep. Length	Fwd. Speed (m/s)
PPO 300k	5.63 ± 6.65	317.6 ± 212.3	0.025 ± 0.006
PPO 1.5M	447.3 ± 42.2	2000 ± 0	0.00045 ± 0.0006

Table 5: Walk-variant vs. original imitation reward (PPO 1.5M, single seed).

Policy	Return	Ep. Length	Fwd. Speed (m/s)
Orig. 1.5M (hacks)	447.3 ± 42.2	2000 ± 0	0.0004 ± 0.0006
Walk 1.5M	83.5 ± 13.0	326.6 ± 31.2	0.116 ± 0.006
Walk 3.5M (cont.)	157.2 ± 0.0	599.0 ± 0.0	0.068 ± 0.0

Reward Ablation: Walk Variant

To test the diagnosis, we defined a *walk* environment variant with two targeted changes: (i) the forward gate no longer requires single-support contact, so upright standing robots still receive forward reward if they actually move; and (ii) we reweighted the reward to emphasize forward terms ($r_{\text{fwd-vel}}$: $0.28 \rightarrow 0.45$, $r_{\text{fwd-prog}}$: $0.24 \rightarrow 0.35$) and de-emphasize posture and in-place matching ($r_{\text{imitation}}$: $0.07 \rightarrow 0.03$, r_{height} : $0.08 \rightarrow 0.06$, r_{upright} : $0.08 \rightarrow 0.06$). We trained one PPO seed for 1.5M steps on this variant. Results (Table 5, Figure 3) show that the stand-still local optimum is broken and that the policy actually walks: forward speed rises from 4.5×10^{-4} m/s to **0.116 m/s** (more than a $250\times$ improvement, and roughly 58% of the commanded target $v_x^* = 0.2$ m/s). Episode length drops from 2000 to ~ 327 because stable long-horizon locomotion has not yet converged—the agent walks, then falls—but this is a qualitatively different failure mode from the stand-still one, and a 58%-of-target speed is a visibly walking gait.

The take-away is not that one reward is right and another is wrong, but that multi-term shaped locomotion rewards sit in a narrow valley between two failure modes: too much posture weight collapses to a stand-still local optimum; too much forward-progress weight destabilizes posture before the agent can learn balanced motion. We did not find a single-shot reward specification that produced a stable walking gait within 1.5M CPU-budget timesteps; larger compute and reference-motion tracking (Peng et al. 2018) are natural next steps.

Extending the ablation to all four algorithms. To test whether the walk-reward improvement generalizes beyond PPO, we additionally trained SAC, A2C, and TQC for 300,000 steps (single seed) on the walk variant (Table 6, Figure 4). The result reinforces the main finding of this project: the two on-policy methods (PPO and A2C) both learn to *move forward*, while the two off-policy methods (SAC and TQC) instead drift *backward* (-0.035 and -0.022 m/s respectively)—they learn some locomotion in the wrong direction, presumably because their replay buffers accumulate short backward-slip transitions that happen to avoid immediate termination. TQC achieves the highest return (28.7)

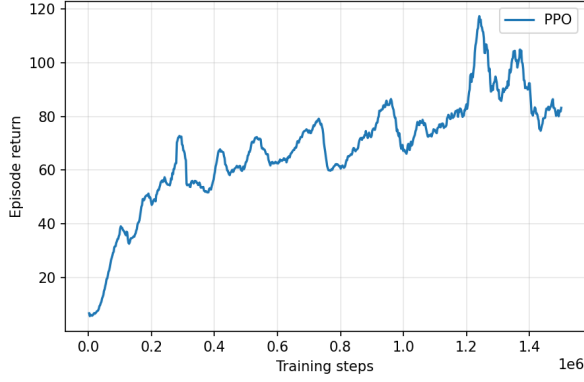


Figure 3: Training return on the walk ablation reward (single seed, PPO, 1.5M steps). Return grows steadily as the agent learns to move forward without falling immediately.

Table 6: All four algorithms on the walk variant (300k, single seed). On-policy methods move forward; off-policy methods drift backward.

Algo	Return	Ep. Length	Fwd. Speed (m/s)
A2C	10.1 ± 0.0	195.0 ± 0.0	$+0.096 \pm 0.0$
SAC	4.0 ± 0.0	169.0 ± 0.0	-0.035 ± 0.003
TQC	28.7 ± 1.7	264.3 ± 8.1	-0.022 ± 0.0
PPO (1.5M, ref.)	83.5 ± 13.0	326.6 ± 31.2	$+0.116 \pm 0.006$

despite moving backward, which once more illustrates that in this task “return” is a weak proxy for “walking”. A2C’s 0.096 m/s at only 300k steps is within a factor of 1.2 of PPO’s 0.116 m/s at 1.5M, confirming that on-policy methods are the correct fit for this task regime.

Limitations. The walking policy is qualitatively correct—it lifts one foot at a time, swings it forward, plants it, and shifts weight—but quantitatively brittle: across 10 evaluation episodes, the agent typically takes 2–3 visible steps (~ 0.3 m of forward motion over ~ 2.6 s) before losing balance and tripping, which is why mean episode length sits at ~ 327 steps rather than the 2000-step cap. The failure mode is consistent: the agent has learned the *open-loop* gait pattern (swing/stance/phase-switching) but has not learned to use IMU/contact feedback for recovery, so a small accumulating error in footstep placement eventually causes the base to lean past the termination pitch/roll threshold. A video sample of the walking-then-falling behavior is available at <https://youtu.be/DXRrKvcgk8M>.

Continued Training (3.5M). Warm-starting from the 1.5M walk checkpoint, we ran PPO for another 2M steps on the same walk reward. The resulting policy traded speed for stability: mean episode length rose from 327 to **599** steps (+83%, i.e. the robot walks for ~ 4.8 s before falling, vs. ~ 2.6 s at 1.5M), while mean forward speed dropped from 0.116 to 0.068 m/s; total forward distance per episode is approximately unchanged (~ 0.3 m). The learning curve (Fig-

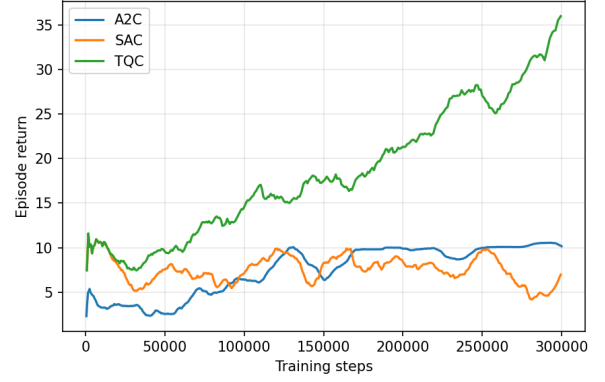


Figure 4: Training return on the walk variant for the three non-PPO algorithms (single seed, 300k steps each). TQC achieves the highest return but its policy walks *backward*; A2C’s return is modest but it actually moves forward at 0.096 m/s (see Table 6).

ure 5) however is non-monotonic: return peaks around 125 near the 1.95M-step mark and then gradually drifts down to ~ 65 by 3.5M. This reflects a well-known PPO failure mode during long continued training: as the clipped update accumulates across many iterations the policy oscillates around a locally optimal gait, and without intermediate checkpointing the final policy is not necessarily the best encountered. Mitigations include learning-rate decay, periodic best-so-far checkpointing, and early-stopping on evaluation metrics rather than training return—all standard in production RL pipelines but outside the scope of this project’s SB3-default protocol.

When Does Each Algorithm Succeed or Fail?

PPO succeeds because (a) on-policy updates always reflect the current regime (falling early, tottering mid-training, walking late), avoiding stale replay targets; and (b) clipped updates provide conservative improvement that is forgiving of the dense, multi-term shaped reward. PPO fails (insufficient speed) when the forward gate g rarely opens during early training—the gradient then only shapes posture, not locomotion. **SAC fails** primarily because its replay buffer is dominated by short falling episodes in the first 50k steps; the critic learns pessimistic Q-values on recoverable states, and automatic entropy tuning keeps the actor exploring wildly, further extending the fall regime. **A2C** partially succeeds but has extremely high seed variance due to very short $n_{\text{steps}} = 5$ advantages. **TQC** behaves similarly to SAC; the truncated distributional critic reduces overestimation but still suffers from the contact-regime distribution shift in the replay buffer.

Discussion

Although SAC is frequently promoted for sample efficiency, this project’s CPU-limited budget, contact-rich dynamics, and heavily shaped reward favored PPO. The reward-engineering observations reinforce a recurring lesson: dense

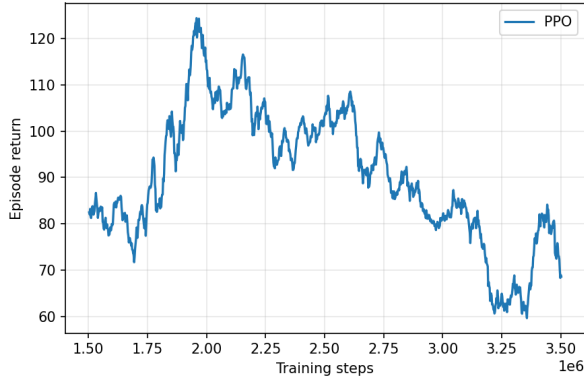


Figure 5: Continued training from the 1.5M walk checkpoint for an additional 2M steps. Return peaks near 1.95M and then drifts down, illustrating PPO’s long-horizon oscillation; the final policy walks more stably (len=599) but more slowly (0.068 m/s) than the 1.5M checkpoint.

multi-term rewards need careful numerical scaling, and a single term silently reduced to a constant can mask learning failure for thousands of training steps. Matching reward scale, then running PPO, then only afterwards experimenting with off-policy methods, is a robust practical order on this kind of task.

Conclusion

Under matched budgets in the Open Duck Mini locomotion task, PPO clearly outperforms SAC in return, episode length, and forward speed; A2C similarly dominates TQC in the supplementary comparison. Beyond the algorithmic comparison, we identified concrete reward-function bugs whose fix was a precondition for any algorithm to produce nonzero forward progress. Our scaling study further uncovers a reward-hacking pathology that is invisible at short budgets: at 1.5M steps PPO converges to a stand-still policy that is near-optimal for the given reward but trivial as a walking controller. A reward ablation (walk variant) confirms the mechanism and recovers a visibly walking gait at 58% of the commanded target speed. Extending the ablation to all four algorithms shows that on-policy PPO and A2C both move forward while off-policy SAC and TQC drift backward, reinforcing the main PPO > SAC finding. This reinforces a practical lesson: for heavily shaped locomotion rewards, longer training is a magnifier, not a fix, for reward-specification error. Future work: longer training with GPU acceleration; massively parallel rollouts (Rudin et al. 2022); and reference-motion tracking (Peng et al. 2018) with ONNX-distilled pretraining from the project’s released walking policy.

Reproducibility

All runs live under runs/: main comparison in main_ppo_sac.300k, supplementary in supp_a2c_tqc.150k. Scripts: train_compare.py, evaluate_compare.py,

plot_learning_curves.py. Reward-fix diffs are in the v2 git branch.

References

- Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *International Conference on Machine Learning*.
- Kuznetsov, A.; Shvechikov, P.; Grishin, A.; and Vetrov, D. 2020. Controlling Overestimation Bias with Truncated Mixture of Continuous Distributional Quantile Critics. In *International Conference on Machine Learning*.
- Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; and Kavukcuoglu, K. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *International Conference on Machine Learning*.
- Peng, X. B.; Abbeel, P.; Levine, S.; and van de Panne, M. 2018. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Transactions on Graphics*, 37(4).
- Pirrone, A.; and Passault, G. 2024. Open Duck Mini. https://github.com/apirrone/Open_Duck_Mini.
- Raffin, A.; Hill, A.; Gleave, A.; Kanervisto, A.; Ernestus, M.; and Dormann, N. 2021. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268): 1–8.
- Rudin, N.; Hoeller, D.; Reist, P.; and Hutter, M. 2022. Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning. In *Conference on Robot Learning*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- Todorov, E.; Erez, T.; and Tassa, Y. 2012. MuJoCo: A Physics Engine for Model-Based Control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033.